

Assignment-10.4

Name-K.Bhavya Sri

H.no-2303a51863

Batch-13

Task1:AI-Assisted Syntax and Code Quality Review

Scenario:You join a development team and are asked to review a junior developer's python script that fails to run correctly due to basic coding mistakes.Before deployment,the code must be corrected and standardized.

Task Description:You are given a Python Script containing:

- Syntax errors
- Indentation issues
- Incorrect variable names
- Faulty function calls

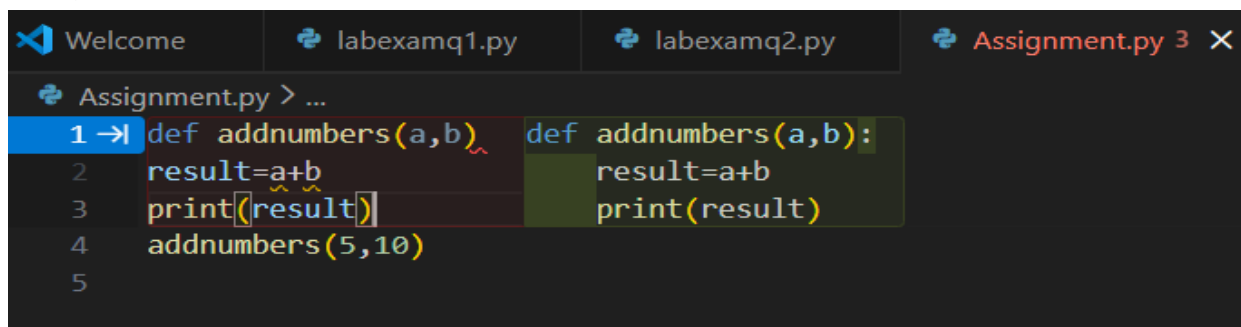
Use an AI tool (GitHub Copilot / Cursor AI) to:

- Identify all syntactic and structural errors
- Correct them systematically
- Generate an explanation of each fix made

Expected Outcome

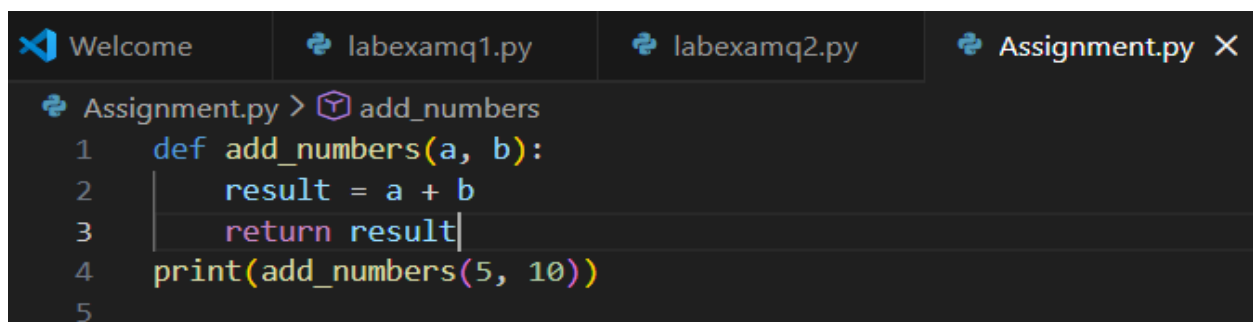
- Fully corrected and executable Python code
- AI-generated explanation describing:
 - o Syntax fixes
 - o Naming corrections
 - o Structural improvements
- Clean, readable version of the script

Faulty code:



```
1 → def addnumbers(a,b)
2   result=a+b
3   print([result])
4   addnumbers(5,10)
5
```

Corrected code:



```
1 def add_numbers(a, b):
2     result = a + b
3     return result
4 print(add_numbers(5, 10))
5
```

Outcome-AI Explanation of Fixes:

- 1) Added missing colon to fix syntax error
- 2) Corrected indentation to follow python rules
- 3) Fixed variable name mismatch for consistency
- 4) Used return instead of printing inside the function
- 5) Improved function name using snake_case

Task2:Performance-Oriented Code Review

Scenario: A data processing function works correctly but is inefficient and slows down the system when large datasets are used.

Task Description: You are provided with a function that identifies duplicate values in a list using inefficient nested loops.

Using AI-assisted code review:

- Analyze the logic for performance bottlenecks
- Refactor the code for better time complexity
- Preserve the correctness of the output

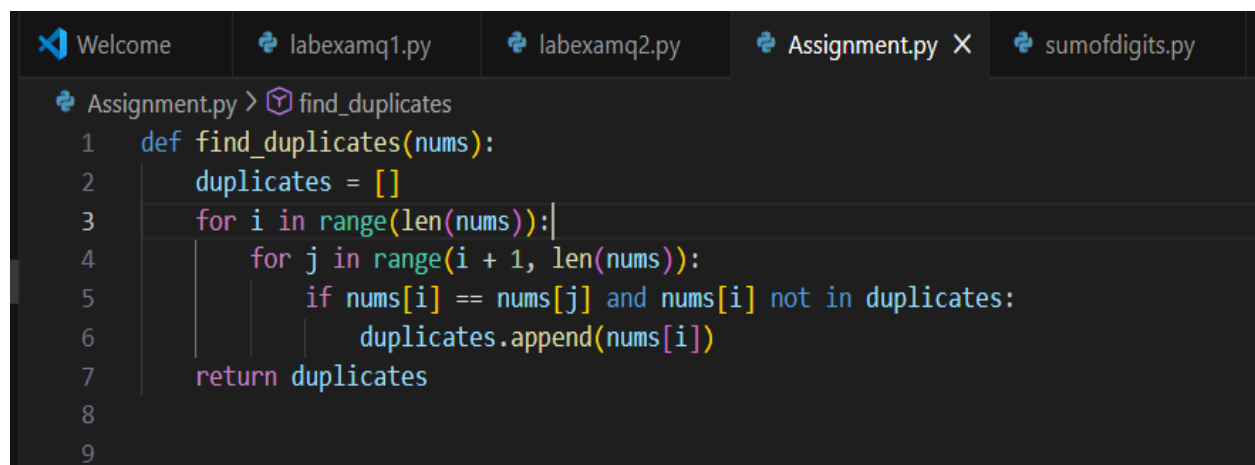
Ask the AI to explain:

- Why the original approach was inefficient
- How the optimized version improves performance

Expected Outcome

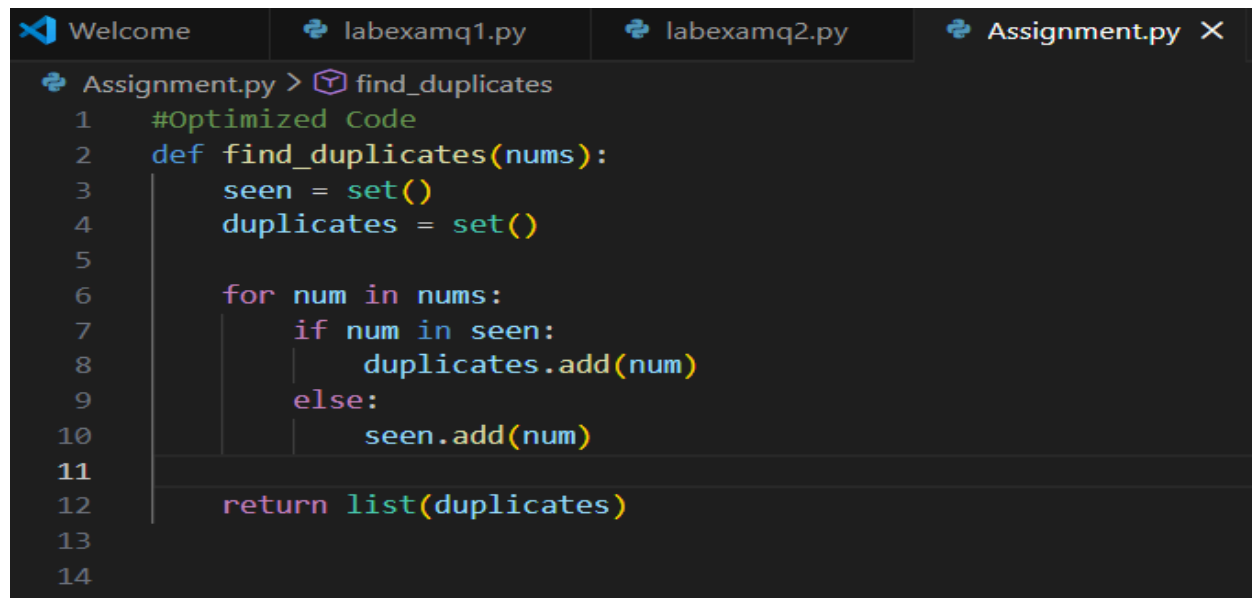
- Optimized duplicate-detection logic (e.g., using sets or hash-based structures)
- Improved time complexity
- AI explanation of performance improvement
- Clean, readable implementation

Inefficient Code:



```
Assignment.py > find_duplicates
1 def find_duplicates(nums):
2     duplicates = []
3     for i in range(len(nums)):
4         for j in range(i + 1, len(nums)):
5             if nums[i] == nums[j] and nums[i] not in duplicates:
6                 duplicates.append(nums[i])
7     return duplicates
8
9
```

Optimized Code:



```
Welcome  labexamq1.py  labexamq2.py  Assignment.py X
Assignment.py > find_duplicates
1  #Optimized Code
2  def find_duplicates(nums):
3      seen = set()
4      duplicates = set()
5
6      for num in nums:
7          if num in seen:
8              duplicates.add(num)
9          else:
10             seen.add(num)
11
12     return list(duplicates)
13
14
```

Outcome-AI Explanation of performance Improvement:

- 1) Replaced nested loops with a single loop.
- 2) Used sets for fast lookup ($O(1)$)
- 3) Reduced time complexity from $O(n^2)$ to $O(n)$
- 4) Improves scalability for large datasets

Task3:Readability and Maintainability Refactoring

Scenario: A working script exists in a project, but it is difficult to understand due to poor naming, formatting, and structure. The team wants it rewritten for long-term maintainability.

Task Description: You are given a poorly structured Python function with:

- Cryptic function names
- Poor indentation
- Unclear variable naming
- No documentation

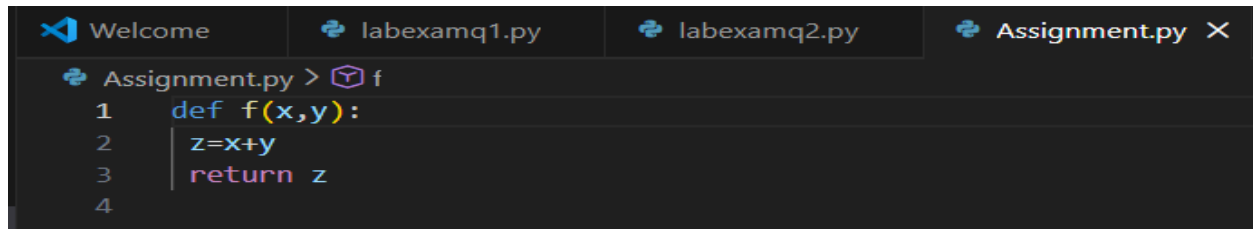
Use AI-assisted review to:

- Refactor the code for clarity
- Apply PEP 8 formatting standards
- Improve naming conventions
- Add meaningful documentation

Expected Outcome

- Clean, well-structured code
- Descriptive function and variable names
- Proper indentation and formatting
- Docstrings explaining the function purpose
- AI explanation of readability improvements

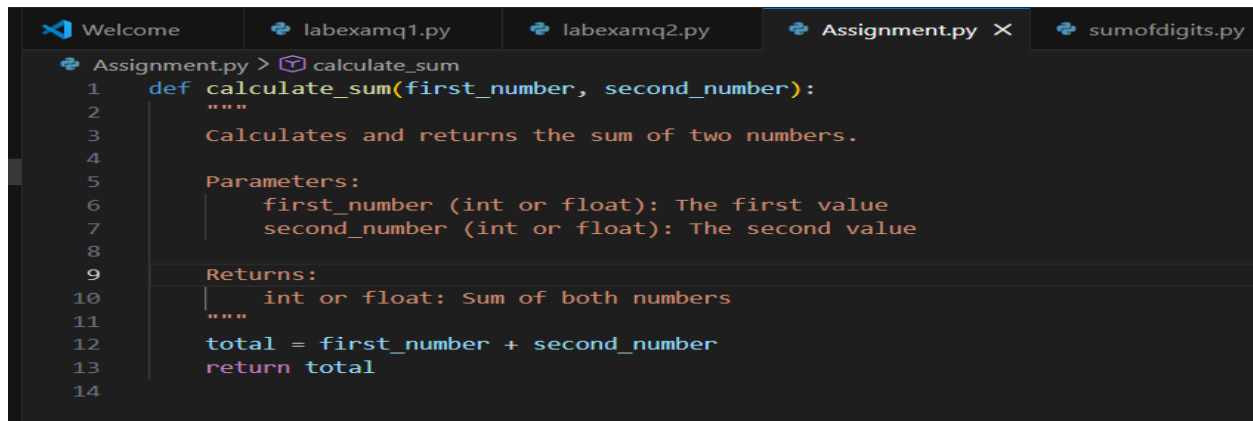
Poorly Structured Python:



The screenshot shows a code editor with four tabs: 'Welcome', 'labexamq1.py', 'labexamq2.py', and 'Assignment.py'. The 'Assignment.py' tab is active, showing a Python function `f(x,y)` with the following code:

```
1 def f(x,y):
2     z=x+y
3     return z
4
```

Refactored Code(PEP8 Compliant):



The screenshot shows a code editor with five tabs: 'Welcome', 'labexamq1.py', 'labexamq2.py', 'Assignment.py', and 'sumofdigits.py'. The 'Assignment.py' tab is active, showing a Python function `calculate_sum` with the following code:

```
1 def calculate_sum(first_number, second_number):
2     """
3     Calculates and returns the sum of two numbers.
4
5     Parameters:
6     first_number (int or float): The first value
7     second_number (int or float): The second value
8
9     Returns:
10    int or float: Sum of both numbers
11    """
12    total = first_number + second_number
13    return total
14
```

Outcome-AI Explanation of Improvements:

- 1)Used descriptive function and variable names
- 2)Applied proper indentation and spacing
- 3)Added docstring for documentation
- 4)Improved readability and future maintainability

Task4:Secure Coding and Reliability Review

Scenario:A backend function retrieves user data from a database but has security vulnerabilities and poor error handling, making it unsafe for production deployment.

Task Description:You are given a Python script that:

- Uses unsafe SQL query construction
- Has no input validation
- Lacks exception handling

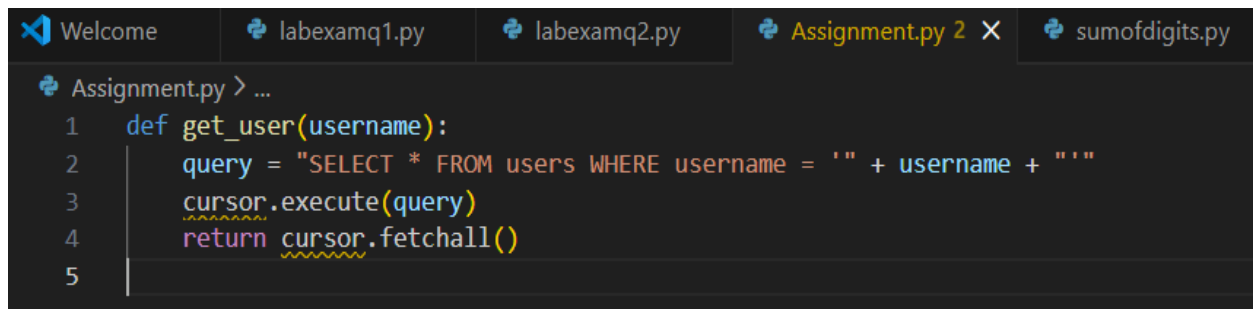
Use AI tools to:

- Identify security vulnerabilities
- Refactor the code using safe coding practices
- Add proper exception handling
- Improve robustness and reliability

Expected Outcome

- Secure SQL queries using parameterized statements
- Input validation logic
- Try-except blocks for runtime safety
- AI-generated explanation of security improvements
- Production-ready code structure

Unsafe SQL query construction(Python script)



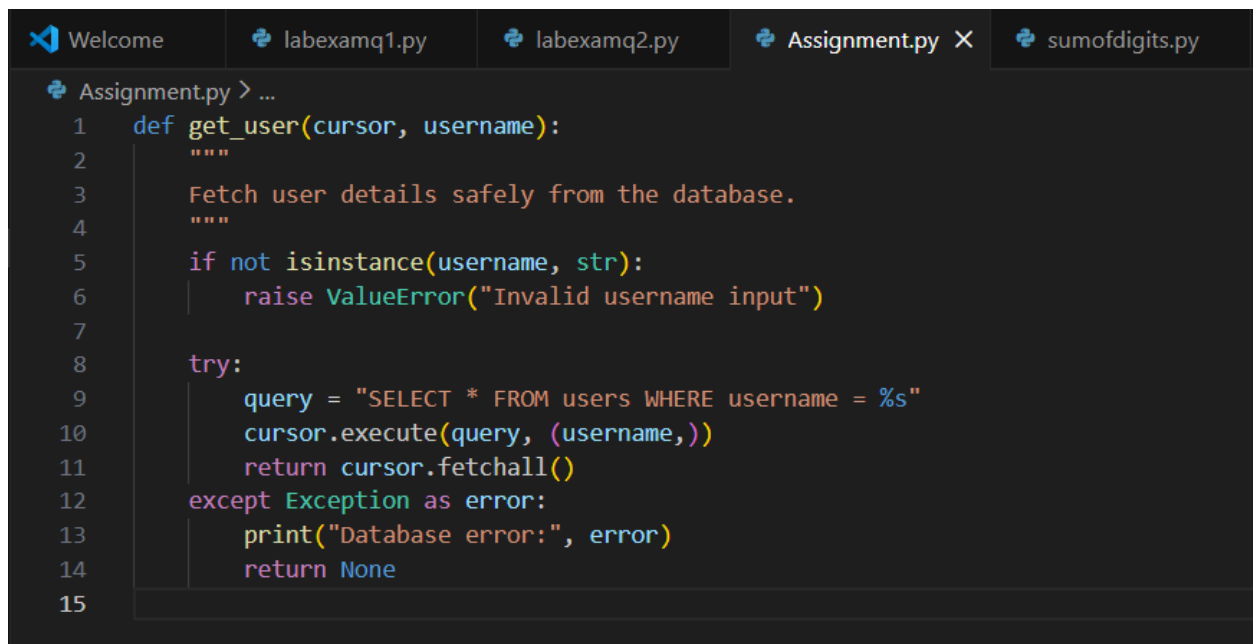
The screenshot shows a code editor with several tabs: 'Welcome', 'labexamq1.py', 'labexamq2.py', 'Assignment.py 2 X', and 'sumofdigits.py'. The 'Assignment.py' tab is active, showing a Python function `get_user(username):`. The function constructs a SQL query string by concatenating the user input directly into the query: `query = "SELECT * FROM users WHERE username = '" + username + "'"`. It then executes the query using `cursor.execute(query)` and returns the results with `cursor.fetchall()`. There is no input validation or exception handling.

```
1 def get_user(username):
2     query = "SELECT * FROM users WHERE username = '" + username + "'"
3     cursor.execute(query)
4     return cursor.fetchall()
5
```

Explanation of Security Issues:

- 1)Vulnerable to SQL Injection
- 2)No input validation
- 3)No exception handling

Secure and Reliable Code:



The screenshot shows the same code editor with the 'Assignment.py' tab active. The code has been updated to be secure and reliable. It includes input validation using `isinstance(username, str)` and raises a `ValueError` for invalid input. The SQL query is now parameterized using `%s` and the `username` variable. The code is wrapped in a `try-except` block to handle database errors, printing the error message and returning `None`.

```
1 def get_user(cursor, username):
2     """
3     Fetch user details safely from the database.
4     """
5     if not isinstance(username, str):
6         raise ValueError("Invalid username input")
7
8     try:
9         query = "SELECT * FROM users WHERE username = %s"
10        cursor.execute(query, (username,))
11        return cursor.fetchall()
12    except Exception as error:
13        print("Database error:", error)
14        return None
15
```

Outcome-AI Explanation of Security Improvements:

- 1)Used parameterized SQL queries to prevent SQL injection
- 2)Added input validation
- 3)Implemented try-except block for error handling
- 4)Improved reliability for production use

Task5:AI-Based Automated Code Review Report

Scenario:Your team uses AI tools to perform automated preliminary code reviews before human review, to improve code quality and consistency across projects.

Task Description:You are provided with a poorly written Python script.

Using AI-assisted review:

- Generate a structured code review report that evaluates:

- o Code readability
- o Naming conventions
- o Formatting and style consistency
- o Error handling
- o Documentation quality
- o Maintainability

The task is not just to fix the code, but to analyze and report on quality issues.

Poorly written python Script:

The screenshot displays an AI code review interface. On the left, a code editor shows a file named 'Assignment.py' with a function 'cal' defined as follows:

```
1 def cal(a,b):return a/b
2
```

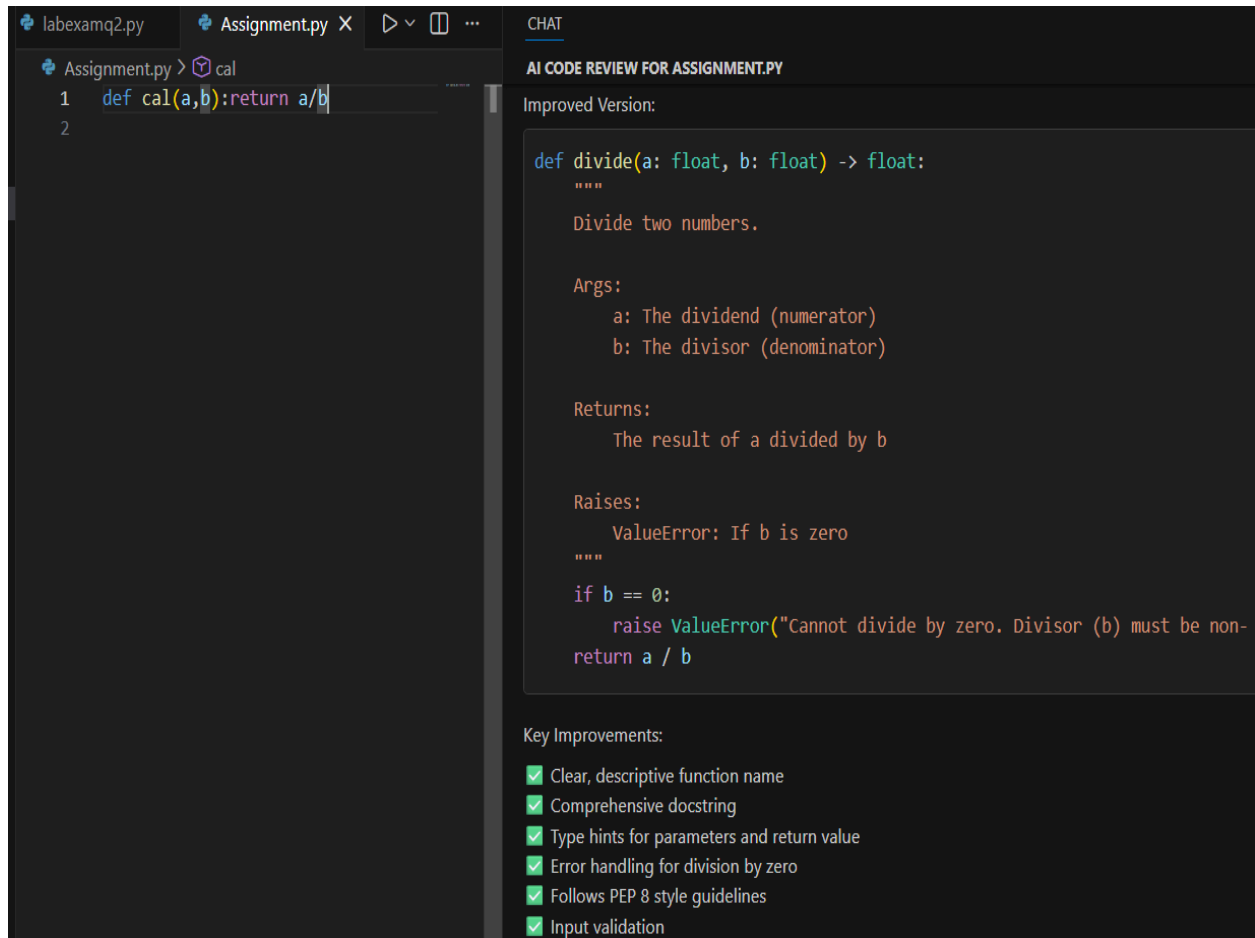
On the right, the 'CHAT' panel shows the 'AI CODE REVIEW FOR ASSIGNMENT.PY' report. The report title is 'Code Review for Assignment.py'. It begins with 'Here's my analysis of your code:' and shows the code snippet: `def cal(a,b):return a/b`.

The 'Issues Found:' section lists five items:

- 1. Poor Function Naming** (marked with a red X):
 - o `cal` is vague and doesn't clearly indicate the function performs division
 - o **Suggestion:** Use `divide()`, `calculate_division()`, or `divide_numbers()`
- 2. Missing Documentation** (marked with a red X):
 - o No docstring explaining what the function does, parameters, or return value
 - o **Suggestion:** Add a docstring with parameter and return type descriptions
- 3. No Type Hints** (marked with a red X):
 - o No indication of expected parameter types
 - o **Suggestion:** Add type hints for clarity and IDE/type checker support
- 4. No Error Handling** (marked with a yellow triangle):
 - o **Critical:** Will crash with `ZeroDivisionError` if `b = 0`
 - o No validation or exception handling
 - o **Suggestion:** Add a check for zero division or handle the exception
- 5. PEP 8 Style Violation** (marked with a yellow triangle):
 - o Function definition and return statement on same line violates Python style guidelines
 - o **Suggestion:** Format across multiple lines for readability

Below the issues, the 'Improved Version:' section is partially visible.

Improved Version:



Conclusion: This lab demonstrated how AI tools can:

- 1) Detect syntax and logical errors
- 2) Improve performance and scalability
- 3) Enhance readability and maintainability
- 4) Identify security vulnerabilities
- 5) Acts as an effective automated code reviewer

AI-Assisted code review significantly improves software quality before human review.